The Relay: Final Report (DRAFT: placeholders remain)

Labels: **[REAL]** measured on Groq. **[SIM]** stand-in simulation. **[DOC]** provider documentation. **[ASSUMED]** unverified assumption. **[TODO]** needs the author. All numbers come from `results/results.csv` (896 rows: 5 real, 891 stand-in) via `python -m scripts.generate_report`.

1. What I built and why

A pipeline that answers questions from three arXiv papers only: validate, classify the question, retrieve through a `search_papers` tool (BM25, optional hybrid), then a sufficiency **branch** (one query-expansion **loop**, else refuse with zero model calls), generate with page citations, check citations deterministically, verify with a second model call, one correction loop, else refuse (at most 4 model calls). Calls go through a gateway with a bounded priority queue, token-bucket limiter, admission control and a retry policy. The decision that shaped it most: the Groq limit that binds is tokens per minute (8,000 [REAL header]), not request count. At about 1,570 tokens per call that is roughly 5 calls per minute (arithmetic on a header value and the stand-in workload), and an answer takes two calls. So the gateway reserves tokens before sending and sheds early instead of queuing long. Details: `docs/DECISIONS.md`.

2. The real service and my stand-in

Table 1 (from `report/tables.md`):

Limit	Value found	How it was found	Confidence
Requests per minute	30 documented; not observed	Groq docs (rate-limits page, 2026-11-10). No response header exposes RPM; the models page lists 1,000 (conflict)	low
Tokens per minute	8,000 (limit header, 5/5 responses)	x-ratelimit-limit-tokens in baseline_v2.csv; remaining/reset headers imply a bucket refilling ~133.3 tokens/s	high (limit), medium (refill model)
Requests per day	1,000 (limit header, 5/5 responses)	x-ratelimit-limit-requests; reset header = used x 86.4 s, i.e. continuous refill (inferred)	high (limit), medium (refill model)
Tokens per day	200,000 documented; not observed	Groq docs only; no header exposes the daily token budget; tracked locally by the usage ledger	low
Requests in flight at once	not measured	deliberately not probed (would risk the free-tier quota); stand-in uses an ASSUMED 8	n/a
What it returns when a limit is hit	not observed (0 of 5 baseline requests were limited). Docs: HTTP 429 + retry-after	documented only; run experiments/probe_429.py to observe; stand-in body text is ASSUMED	low
Time until it recovers	inferred from headers: tokens (limit-remaining)/133.3 s; requests 86.4 s per request	arithmetic on baseline_v2.csv reset headers; never observed after a real 429	low-medium

The RPM conflict (30 documented, 1,000 on the models page, no header) is unresolved and not guessed. **Stand-in** (`standin/`): streams a reasoning phase then content, becomes slower under load, refuses with 429 plus `retry-after` when a limit is reached, and can fail. **Calibrated from the 5 real baseline requests** [REAL, n=5]: median time to first reasoning token 1,360 ms (log-sigma 0.274), 2.139 ms per output token, median 269 output tokens (sigma 0.573), about 70 prompt-overhead tokens. **Assumed, not measured**: in-flight limit 8, provider queue 16, load sensitivity 0.5, recovery 5 s, zero random failures, and the 429 body text. **Least like the real service**: calibration rests on only 5 requests; no real 429, concurrency or failure was ever observed.

3. Results

Percentiles are over successful requests only; p99 is interpolated and unstable below 100 successes (†). Only the first row is real [REAL, 5 requests]; all others are [SIM] and exercise the gateway only (no retrieval or verification). Offered rate is requests sent over the window.

Table 2a

Scenario	Real/stand-in	Requests	Duration s	Offered/min	TTFT p50/p95/p99	Total p50/p95/p99	n ok
real-baseline-v2	real	5	24	13.5	2359 / 2616 / 2623†	2359 / 2838 / 2880†	5
steady	stand-in	28	462	3.6	2005 / 2827 / 2940†	2123 / 3031 / 3215†	28
burst	stand-in	32	331	5.8	2532 / 14699 / 22357†	2574 / 14962 / 22576†	17
overload	stand-in	60	319	12.2	7473 / 24442 / 28470†	7528 / 24505 / 28522†	29
limit_halved	stand-in	21	340	3.7	2565 / 19408 / 24130†	2600 / 19504 / 24202†	17
daily_exhausted	stand-in	30	290	6.2	1995 / 2541 / 2560†	2170 / 2725 / 2848†	12

† fewer than 100 successful requests: p99 is interpolated between the largest observations and is not a stable estimate.

Table 2b

Scenario	Source	n	Succeeded	Rate-limited	Timed out / given up	Other error	Retries/request	Peak queue (at submit)	Tokens/answer	Cost/1,000 answers
real-baseline-v2	real	5	100.0%	0.0%	0.0%	0.0%	0.00	not measured	383	$0.10 (hypothetical; free tier billed $0)
steady	stand-in	28	100.0%	0.0%	0.0%	0.0%	0.00	0	1545	$0.17 (simulated)
burst	stand-in	32	53.1%	46.9%	0.0%	0.0%	0.00	13	1545	$0.17 (simulated)
overload	stand-in	60	48.3%	51.7%	0.0%	0.0%	0.00	1	1568	$0.18 (simulated)
limit_halved	stand-in	21	81.0%	19.0%	0.0%	0.0%	0.43	0	1522	$0.16 (simulated)
daily_exhausted	stand-in	30	40.0%	60.0%	0.0%	0.0%	0.00	0	1561	$0.18 (simulated)

Number I could not have guessed: in the real baseline the median gap between the first visible content token and the end of the answer is 15 ms (content TTFT p50 2,359 ms equals total p50 2,359 ms), so about all the wait is hidden reasoning, and `tokens_out` includes those reasoning tokens against the 8,000 TPM budget. **Answer quality on at least 15 questions: not measured** (needs `benchmarks/questions.jsonl` from the real PDFs and a real Groq run). Tokens per answer: 383 real (baseline, one call); about 1,550 per simulated call.

4. Where the time goes

Slowest simulated request, `lt-main-burst-0015` [SIM]: total 24,479 ms; first token at 24,271 ms, only 208 ms after it. Components from gateway timestamps: queue wait 12,016 ms, limiter pacing wait 10,320 ms, retry delay 0 ms, model wait before first token 1,926 ms, generation after first token 208 ms, unattributed 9.5 ms. So the delay was **waiting, before the first token**: 91% of the time was queue plus pacing, caused by a 20-request burst against 4 concurrent slots and the token budget. It sums to the total within 10 ms, which is how I know. No slow *real* request exists (real max total 2,891 ms), so this is a stand-in example.

5. When a limit is hit

Per minute: the limiter reserves estimated tokens first. If the needed wait exceeds the request's deadline it is shed immediately (`admission_wait`, reported `rate_limited`), otherwise it waits. Interactive requests go before batch, so batch is delayed first. A 429 `retry-after` sets a shared cool-down. Nobody gets a weaker model; when the budget is near, the prompt is trimmed to fit (at least 2 passages) or the question is refused with an honest reason. In simulation, overload at about 12 per minute gave 48.3% success and 51.7% rate-limited; burst gave 53.1% and 46.9%. **Per day:** the first daily-exhausted 429 is probed once, then requests fail fast without queuing, since waiting until tomorrow serves nobody; real runs stop at 80% of documented daily budgets (ledger). Simulated with a scaled-down budget of 12 requests: 40.0% succeeded. Real Groq 429 behaviour: **not measured** (`experiments/probe_429.py` measures it).

6. Retrieval: quality versus cost

Three settings are implemented (piece size / overlap / pieces fetched / re-rank): A 300/50/3/no, B 600/100/5/no, C 600/100/5/lexical re-rank (not a cross-encoder). Results (Table 3 is generated; nothing is filled in until measured):

Table 3

Setting (piece size / overlap, pieces fetched, re-rank)	Questions asked	Correct and cited %	Wrong %	Correctly refused %	Tokens per answer	First token p50 / p95 (ms)	Total p50 / p95 (ms)
A: 300/50, 3, no	not measured	not measured	not measured	not measured	not measured	not measured	not measured
B: 600/100, 5, no	not measured	not measured	not measured	not measured	not measured	not measured	not measured
C: 600/100, 5, yes	not measured	not measured	not measured	not measured	not measured	not measured	not measured

[TODO] Chosen setting and why (needs the measurements). When the limit is close, retrieval shrinks the context to the remaining token room (minimum 2 passages) and refuses if even that does not fit.

7. How this library will mislead a naive retriever

Failure mode 1: "chunking" means two different things. The Dynamic Chunking paper's abstract (arXiv abs page, read 2026-11-12) describes a learned mechanism that segments the input sequence, trained with the model so a separate tokenizer is not needed; it is not text splitting for retrieval. A keyword retriever asked a question about choosing a chunk size for retrieval would rank this paper's passages highly on "chunk", and the answer built from them would describe a different technique. **[PENDING: PDF verbatim quote and page; no example passage is cited until it is verified against the PDF.] [TODO]** Failure modes 2 and 3, each with an example question, must be found in the real PDFs (none are written here to avoid inventing paper content).

8. Retries

Policy: jittered exponential backoff, honours `retry-after`, 3 attempts, no retry of invalid or daily-exhausted errors, a retry budget (30% of traffic plus burst 3), backoff sleeps release the concurrency slot. **Where retrying made things worse** [SIM, ASSUMED]: with 40 requests per minute against an RPM of 20 and the assumption that rejected attempts also consume RPM, naive clients (immediate, fixed, exponential, jittered) sent about 5 attempts per request and only about 20% succeeded. The same naive client when rejections cost nothing succeeded 45.8%. The hardened client succeeded 42.5% (19.2% timed out, p50 14,437 ms). Whether Groq counts rejected attempts is **unverified**, so the size of the effect is conditional on it.

Retry strategies [SIM] (retry_* scenarios; rows are in results.csv)

Scenario	n	Succeeded	Rate-limited	Timed out	Provider attempts/request	Attempts per ok answer	OK total p50 ms
retry_exponential	120	20.0%	80.0%	0.0%	5.02	25.1	2312
retry_fixed	120	20.0%	80.0%	0.0%	5.03	25.1	2310
retry_hardened	120	42.5%	38.3%	19.2%	0.43	1.0	14437
retry_jitter	120	20.0%	80.0%	0.0%	5.02	25.1	2319
retry_naive_immediate	120	19.2%	80.8%	0.0%	5.04	26.3	2330
retry_naive_immediate_nocount	120	45.8%	54.2%	0.0%	3.71	8.1	2333

9. What the person sees

At 3pm with the daily budget gone, the person gets immediately, not after waiting: "Today's model capacity is used up, so no answer can be generated now. Please try again later." (HTTP 429, status `rate_limited`). The text gives no reset time, because the reset is a rolling refill inferred from headers, not observed. Questions the papers cannot answer still get a normal refusal with the reason, using no model call.

10. What I'm unsure about, what I left out, what I learned

Least certain assumption: whether rejected requests consume budget, and the real RPM (30 vs 1,000). If wrong, the retry result above shrinks toward the 45.8% control, and the RPM pacing may be too strict or too loose; TPM still binds either way. **Left out** (confirm): a cross-encoder re-ranker (a lexical one is used), and showing the answer to the person before verification (answers are held until checked, which costs time to first visible token). **Learned: [TODO]** one thing new to you, in your own words.

11. Your feedback on Vaani

[TODO] Your own honest view after visiting https://www.vaani.dev/: what stood out, what confused you, one thing you would change.

12. How I worked with the assistant

Real, unedited history: `report/conversation_history.md` (this environment's sessions; export any earlier Windows-side chat and append it). **[TODO]** Your account: the three prompts that mattered most and why (candidates from the history: the first master prompt with its three priorities; the Phase 1 review prompt that required content-only first-token timing and preserving `baseline.csv`; the instruction to never present stand-in results as Groq results), one thing it got wrong that you caught and how, one decision where you overruled it, and what is distinctly yours.